BÖLÜM 8 / MODULE 8

Kubernetes Operators – Kendi Akıllı Robotunu Kodla

Kubernetes Operators – Build Your Own Robot SRE

Bir insan SRE mühendisinin beynini koda dökmek: CRD'ler, sonsuz mutabakat (reconciliation) döngüsü, popüler operatörler, OperatorHub ve Java Operator SDK (JOSDK).

Automating the human Site Reliability Engineer: Custom Resource Definitions (CRDs), the reconciliation loop, production operators, OperatorHub, and building Java operators.

43

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Operator Deseni Nedir? (Bir SRE Mühendisinin Beynini Koda Dökmek)

The Operator Pattern: Automating the Human SRE

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Şirketinizde çalışan kıdemli bir veritabanı uzmanını düşünün: Gece saat 03:00'te ana veritabanı çöktüğünde uyanır, yedek sunucuyu lider yapar, verileri eşitler, eski sunucuyu tamir edip havuza sokar. İnsan yorulur, uyur, hata yapar. Bir 'Operator' ise bu kıdemli mühendisin beynindeki tüm kuralları ve tecrübeyi 7/24 uykusuz çalışan bir yazılım robotuna dönüştürmektir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Picture a veteran database administrator in your company: When the primary database crashes at 3 AM, they wake up, promote the replica to master, synchronize data, and heal the cluster. Humans get sleepy and make typos. An 'Operator' is encapsulating that human engineer's operational wisdom into software code running 24/7 without ever taking a coffee break!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes standart olarak Deployment ve Service gibi temel durumsuz (stateless) iş yüklerini mükemmel yönetir. Ancak PostgreSQL, Kafka veya Redis gibi durum tutan (stateful) sistemler; sürüm yükseltme, otomatik yedekleme (backup), felaketten kurtarma (failover) ve şema göçü gibi alana özgü (domain-specific) karmaşık mantıklar gerektirir. Operator Pattern = Özel Kaynak Tanımı (CRD) + Özel Kontrolcü (Custom Controller).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes manages stateless workloads natively via Deployments. Stateful distributed systems (Postgres, Kafka, RabbitMQ) require domain-specific operational knowledge for backups, failover, and cluster topology negotiations. The Operator Pattern bridges this gap by combining: Custom Resource Definitions (CRD) to store desired state, and a Custom Controller executing reconciliation loops.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Operatörsüz manuel çile (50 adımda Postgres kümesi yönetimi):
# 1. SSH yap, 2. pg_dump al, 3. failover komutu ver...

# Operatör ile tek satırda otomatik küme ve yedekleme:
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: coffee-db-cluster
spec:
  instances: 3 # 3 düğümlü otomatik failover özellikli PostgreSQL!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Her şey için operatör yazmaya kalkmayın! Eğer uygulamanız standart bir REST API veya durumsuz mikroservis ise, standart bir K8s Deployment fazlasıyla yeterlidir. Operatörler karmaşık durum yönetimi (stateful) için tasarlanmıştır.

🇬🇧 WATCH OUT

Do not write operators for simple stateless apps! Standard Deployments are sufficient for 95% of microservices. Reserve operators for complex stateful automation.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Operator Deseni insan uzmanlığını yazılım otomasyonuna dönüştürür.
- Veritabanı yedeği, failover ve küme senkronizasyonunu otonom yönetir.
- CNCF ekosistemindeki en güçlü ve gelişmiş K8s genişletme mimarisidir.

- The Operator pattern translates human operational expertise into automated software.
- Automates complex stateful operations like failover, resizing, and backups.
- The premier architectural pattern for extending the native Kubernetes control plane.

44

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

CRD (Custom Resource Definition) ve Reconciliation Döngüsü

CRDs and the Infinite Reconciliation Loop

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes doğduğunda sadece 'Pod', 'Service' gibi birkaç kelime bilirdi. CRD (Custom Resource Definition), Kubernetes'e yepyeni kelimeler öğretmektir! 'Artık sen `SpringBootApp` veya `CoffeeMachine` kelimesini de bileceksin' deriz. Reconciliation (Mutabakat) Döngüsü ise robotun 'İstenen kahve hazırlandı mı? Eksik var mı?' diye etrafı sonsuz bir merakla kontrol etmesidir.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

When Kubernetes was born, it only understood basic words like 'Pod' and 'Service'. A CRD (Custom Resource Definition) teaches Kubernetes brand-new nouns! You say: 'From now on, you understand what a `SpringBootApp` is.' The Reconciliation Loop is the robot's infinite inner monologue: 'I see 1 desired `SpringBootApp`; do 3 matching pods exist? If not, make it so!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) CRD: K8s API Server'a OpenAPI v3 şemasıyla yeni bir REST uç noktası (`/apis/apps.eli5.io/v1alpha1/...`) ekler. 2) Custom Resource (CR): Bu şablon kullanılarak oluşturulan gerçek nesnedir. 3) Reconciliation Loop: Controller sürekli olarak K8s API'sindeki Watch kuyruğunu dinler. `Reconcile(Request)` fonksiyonu çağrıldığında: Mevcut kaynakları sorgular, aradaki farkı hesaplar ve K8s API çağrılarıyla sistemi olması gereken duruma eşitler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) CRDs register custom REST endpoints into the API Server with OpenAPI v3 schema validation. 2) Custom Resources (CR) are instances conforming to that schema. 3) The Reconciliation Loop triggers on Watch event queues. The `Reconcile(Request)` method retrieves the observed state, calculates the delta against desired state, and issues idempotent API calls to eliminate drift.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# springbootapp-crd.yaml (Yeni Nesne Tanımı)
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: springbootapps.apps.eli5.io
spec:
  group: apps.eli5.io
  versions:
    - name: v1alpha1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                image: { type: string }
                replicas: { type: integer, minimum: 1, maximum: 10 }
                port: { type: integer, default: 8080 }
  scope: Namespaced
  names:
    plural: springbootapps
    singular: springbootapp
    kind: SpringBootApp
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Reconciliation döngüsündeki kodunuz MUTLAKA 'Idempotent' (Tekrarlanabilir) olmalıdır! Döngü 1 kez de çalışsa, peş peşe 100 kez de çalışsa aynı sonucu üretmeli, her seferinde sıfırdan yeni kaynak yaratmamalıdır.

🇬🇧 WATCH OUT

Reconciliation logic MUST be strictly idempotent! Whether executed once or a thousand times, it must evaluate current state and produce the exact same outcome.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- CRD'ler Kubernetes API'sini kendi alan adınıza özel nesnelerle genişletir.
- Reconciliation döngüsü 'İstenen Durum' ile 'Mevcut Durum'u daima eşitler.
- OpenAPI v3 doğrulaması hatalı YAML girişlerini anında reddeder.
- CRDs extend the core Kubernetes API with domain-tailored schema endpoints.
- The reconciliation loop relentlessly drives actual state toward desired state.
- Built-in OpenAPI v3 validation rejects malformed YAML payloads at the API door.

45

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Dünyayı Yöneten Hazır Operatörler: Prometheus, Postgres, Strimzi

Battle-Tested Operators: Prometheus, CloudNativePG, and Strimzi

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Tekerleği baştan icat etmeyin! Dünyanın en iyi mühendisleri zaten en zor işler için mükemmel robotlar yazdı: 1) Prometheus Operator: Kümedeki tüm pod'ları otomatik keşfedip grafiklerini çizer. 2) CloudNativePG: PostgreSQL veritabanını tek satırla kurar, saniyede bir yedek alır, ana sunucu çökerse 3 saniyede yedeği devreye sokar. 3) Strimzi: Gezegenin en karmaşık sistemi olan Apache Kafka'yı çocuk oyuncağına çevirir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Don't reinvent the wheel! The world's top infrastructure architects have already coded elite robots for the hardest problems: 1) Prometheus Operator: Dynamically sniffs out microservices and plots metrics dashboards. 2) CloudNativePG: Deploys HA PostgreSQL, takes automated point-in-time backups to S3, and fails over in 3 seconds flat. 3) Strimzi: Tames Apache Kafka clusters effortlessly!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bu operatörler üretimde altın standarttır: 1) CloudNativePG (`postgresql.cnpg.io`): PostgreSQL streaming replication, PgBouncer bağlantı havuzu, WAL arşivleme ve S3/MinIO yedeklemelerini CRD'lerle tam otomatik yürütür. 2) Prometheus Operator (`monitoring.coreos.com`): `ServiceMonitor` CRD'si ile Spring Boot Actuator `/actuator/prometheus` uç noktasını otomatik olarak Prometheus hedeflerine ekler. 3) Strimzi (`kafka.strimzi.io`): Kafka broker ve Zookeeper/KRaft topolojisini yönetir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Production staples: 1) CloudNativePG manages PostgreSQL physical streaming replication, seamless zero-downtime rolling upgrades, and automated barman-based WAL archiving to S3. 2) Prometheus Operator utilizes `ServiceMonitor` CRDs to auto-discover Spring Boot `/actuator/prometheus` endpoints. 3) Strimzi manages Kafka brokers, topics, and users purely via declarative YAML.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# servicemonitor-spring.yaml (Prometheus için Spring Boot İzleme Kuralı)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: spring-boot-monitor
  namespace: coffee-shop
spec:
  selector:
    matchLabels:
      app: coffee-app
  endpoints:
    - port: http
      path: /actuator/prometheus
      interval: 15s # Her 15 saniyede bir JVM metriklerini topla!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Operatör kurarken CRD'lerin küme geneli (Cluster-wide) yetki gerektirdiğini unutmayın! Standart bir namespace geliştiricisi tek başına CRD kuramaz; cluster-admin yetkisi gerekir.

🇬🇧 WATCH OUT

Installing operators requires cluster-admin privileges because CRD installation is a cluster-scoped operation affecting all tenants.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Popüler açık kaynak operatörler yüzlerce saatlik operasyonel emeği kurtarır.
- `ServiceMonitor` nesnesi Spring Boot metriklerini otomatik Prometheus'a bağlar.
- Veritabanı felaket senaryoları insan hatası olmadan otomatik çözülür.

- Battle-tested open-source operators save hundreds of hours of maintenance toil.
- ServiceMonitors auto-scrape Spring Boot Micrometer metrics without static configs.
- Automated database failover eliminates catastrophic human mistakes during outages.

46

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

OperatorHub ve Operator Lifecycle Manager (OLM)

OperatorHub & Operator Lifecycle Manager (OLM): The K8s App Store

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Akıllı telefonunuza nasıl App Store veya Google Play'den tek tıkla uygulama indiriyorsanız, Kubernetes kümenize de 'OperatorHub' üzerinden tek tıkla kurumsal operatörler yüklersiniz! OLM (Operator Lifecycle Manager) ise telefonunuzun arka plandaki otomatik güncelleme yöneticisidir: Operatörlerin yeni sürümleri çıktığında onları kesintisiz günceller!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Just like browsing the App Store or Google Play to install apps with a single tap, 'OperatorHub' is the official enterprise app marketplace for Kubernetes! OLM (Operator Lifecycle Manager) is the automated app manager running under the hood: when a database operator releases a security patch, OLM upgrades it smoothly without breaking a sweat!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

OLM mimarisi 4 temel kaynaktan oluşur: 1) `CatalogSource` (Mevcut operatörlerin ve sürümlerin listelendiği depo), 2) `Subscription` (Hangi operatöre abone olunduğunu ve güncelleme kanalını - channel: stable/fast - belirtir), 3) `InstallPlan` (Gerekli CRD'leri ve RBAC izinlerini hesaplayan yürütme planı), 4) `ClusterServiceVersion` (CSV: Operatörün tüm manifestolarını, konteyner imajlarını ve metaverilerini paketleyen ana nesne). OpenShift'te yerleşik gelir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OLM manages operator lifecycles via: 1) `CatalogSource` (registry of operators and metadata), 2) `Subscription` (tracks an operator release channel like `stable-v1`), 3) `InstallPlan` (dependency resolution engine generating RBAC and CRD installation steps), 4) `ClusterServiceVersion` (CSV, encapsulated metadata detailing container images and CRDs). OLM is natively built into OpenShift.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# subscription-cloudnative-pg.yaml
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: cloudnative-pg-subscription
  namespace: operators
spec:
  channel: stable
  name: cloudnative-pg
  source: operatorhubio-catalog
  sourceNamespace: olm
  installPlanApproval: Automatic # Güvenlik yamalarını otomatik uygula!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eğer `installPlanApproval: Automatic` seçerseniz, operatör yeni bir ana sürüme (major version) geçtiğinde otomatik güncellenebilir ve bu durum geriye dönük uyumsuzluk yaratabilir. Üretim ortamında `Manual` onay tavsiye edilir.

🇬🇧 WATCH OUT

Setting `installPlanApproval: Automatic` in production can inadvertently trigger breaking major version upgrades. Choose `Manual` approval for enterprise clusters.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- OperatorHub binlerce onaylı ve sertifikalı operatörü tek çatı altında sunar.
- OLM bağımlılıkları çözer, yükseltmeleri yönetir ve operatör sağlığını izler.
- OpenShift Web Konsolunda görsel bir pazar yeri olarak doğrudan entegredir.

- OperatorHub curates thousands of verified, certified community and enterprise operators.
- OLM automates complex operator installations, dependency resolution, and upgrades.
- Exposes a seamless visual marketplace directly inside the OpenShift Web Console.

47

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Java ile Operator Yazımı: Java Operator SDK (JOSDK) Temelleri

Building Operators in Java: Java Operator SDK (JOSDK)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dünyadaki herkes size 'Kubernetes operatörleri sadece Go (Golang) ile yazılır' diye yalan söyleyebilir. İnanmayın! Bir Java ve Spring Boot geliştiricisi olarak, kendi en iyi bildiğiniz dilde K8s operatörü yazabilirsiniz. Java Operator SDK (JOSDK) sayesinde tek yapmanız gereken bir Java arayüzünü (`Reconciler<SpringBootApp>`) uygulamak ve içine kodunuzu yazmaktır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The tech world might tell you that Kubernetes operators can ONLY be coded in Go. Don't believe the myth! As a Java and Spring Boot engineer, you can build production-grade operators in your favorite language. With Java Operator SDK (JOSDK), you simply implement a clean Java interface (`Reconciler<SpringBootApp>`) and write standard object-oriented code!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Java Operator SDK (JOSDK), Fabric8 Kubernetes Client üzerine kurulmuş modern bir framework'tür. Quarkus veya Spring Boot ile entegre çalışır. `@ControllerConfiguration` anotasyonuyla CRD'ye bağlanır. Reconciler arayüzü tek bir metod içerir: `UpdateControl<SpringBootApp> reconcile(SpringBootApp resource, Context<SpringBootApp> context)`. JOSDK arka planda thread havuzunu, Watch eventlerini, retry mekanizmalarını ve exponential backoff algoritmalarını otomatik yönetir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Java Operator SDK (JOSDK) is a high-performance framework built atop Fabric8 Kubernetes Client, integrating seamlessly with Spring Boot and Quarkus. By implementing `Reconciler<T>` and annotating with `@ControllerConfiguration`, JOSDK manages informers, work queues, event filtering, and exponential backoff retry semantics transparently.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
// Java Operator Reconciler İskeleti:
@ControllerConfiguration(name = "springbootapp-controller")
public class SpringBootAppReconciler implements Reconciler<SpringBootApp> {

    private final KubernetesClient client;

    public SpringBootAppReconciler(KubernetesClient client) {
        this.client = client;
    }

    @Override
    public UpdateControl<SpringBootApp> reconcile(SpringBootApp resource,
Context<SpringBootApp> context) {
        String appName = resource.getMetadata().getName();
        String targetImage = resource.getSpec().getImage();
        int replicas = resource.getSpec().getReplicas();

        // Akıllı Robot Mantığı: Deployment var mı kontrol et, yoksa yarat!
        reconcileDeployment(resource, appName, targetImage, replicas);

        return UpdateControl.noUpdate();
    }
}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Java ile operatör yazarken Docker imajınızın bellek tüketimine dikkat edin. Standart bir HotSpot JVM operatör için 200MB RAM harcayabilir. GraalVM Native Image derlemesi yaparak operatörünüzü 20MB RAM ile 0.01 saniyede açılan bir ikili dosyaya dönüştürebilirsiniz!

🇬🇧 WATCH OUT

Standard JVM runtime footprints hover around 200MB. For optimal lightweight operator footprints, compile your Java operator with GraalVM Native Image.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Java geliştiricileri Go öğrenmek zorunda kalmadan yerel K8s operatörleri yazabilir.
- Fabric8 client K8s API'sindeki tüm nesneleri tip güvenli (type-safe) Java sınıflarına çevirir.
- Quarkus ve Spring Boot entegrasyonu modern geliştirme konforu sunar.

- Java engineers can build high-performance K8s operators without mastering Go.
- Fabric8 provides type-safe Java POJOs for all native and custom K8s resources.
- Seamless integration with Spring Boot enables dependency injection and metrics.

48

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Pratik Örnek: `SpringBootApp` CRD'sini Yöneten Mini Java Operator

Hands-on: Building a `SpringBootApp` Custom Operator in Java

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kendi canlı robotumuzu sahaya sürme vakti! Bir yazılımcı sadece 5 satırlık minicik bir `SpringBootApp` YAML'ı oluşturacak. Bizim Java operatörümüz bunu anında havada yakalayacak; arka planda otomatik olarak tam teşekküllü bir Deployment, bir Service ve bir Ingress oluşturup ayaklandıracak! Yazılımcı 100 satır karmaşık YAML yazmaktan kurtulacak!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Time to unleash our live robot onto the cluster! A developer submits an ultra-simple 5-line `SpringBootApp` YAML. Our Java Operator intercepts it instantly and automatically generates a full-blown Deployment, ClusterIP Service, and Ingress route under the hood! Developers are spared from writing 150 lines of boilerplate YAML forever!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Operatörün çalışma prensibi: 1) Kullanıcı `kubectl apply -f sample-springbootapp.yaml` ile CR oluşturur. 2) API Server watch eventi tetikler, JOSDK `reconcile` metodunu çağırır. 3) Operatör `client.apps().deployments().inNamespace(...)` ile Deployment'ın varlığını kontrol eder. 4) Yoksa `new DeploymentBuilder()...` akıcı (fluent) API'siyle Deployment oluşturur. 5) `ownerReference` ekleyerek 'sahiplik' kurar; böylece CR silindiğinde açılan tüm deployment ve servisler K8s Garbage Collector tarafından otomatik temizlenir!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Execution lifecycle: 1) User submits a `SpringBootApp` custom resource. 2) JOSDK dispatches a reconciliation cycle. 3) The reconciler checks for existing child resources via the Fabric8 fluent builder DSL. 4) If absent, it synthesizes matching Deployments and Services. 5) By setting `OwnerReferences`, the operator ensures that deleting the parent CR triggers automated garbage collection of all spawned resources.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# Kullanıcının Yazdığı 5 Satırlık Rüya YAML:
apiVersion: apps.eli5.io/v1alpha1
kind: SpringBootApp
metadata:
  name: my-coffee-shop
  namespace: coffee-shop
spec:
  image: "ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0"
  replicas: 3
  port: 8080
  envMessage: "Operator tarafından yönetilen harika kahve servisi!"

# Operatör bunu yakalar ve arkada Deployment + Service + Ingress'i OTOMATİK kurar!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Alt kaynakları oluştururken mutlaka `OwnerReference` bağlamalısınız! Bağlamazsanız, kullanıcı ana `SpringBootApp` nesnesini sildiğinde arka plandaki Deployment'lar silinmez, kümede 'hayalet iş yükleri' (orphan resources) olarak kalır.

🇬🇧 WATCH OUT

Always attach `OwnerReference` to generated resources! Without owner references, child deployments become orphaned when the custom resource is deleted.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Operatörler karmaşık çoklu kaynak üretimini tek bir basitleştirilmiş CRD'ye indirir.
- `OwnerReference` ile alt kaynakların yaşam döngüsü otomatik bağlanır ve temizlenir.
- Yazılımcı üretkenliği ve platform mühendisliği standartları tavan yapar.

- Operators condense sprawling multi-resource manifests into simple, elegant CRDs.
- OwnerReferences guarantee clean, cascade garbage collection upon deletion.
- Supercharges internal developer platforms (IDP) and engineering team velocity.